ESP32 MDK PortaPack Emulator Portierung

https://gemini.google.com/app/b28626ab10978d78

User prompt: Ich möchte https://github.com/D4C1-Labs/ARM-Cortex-M3-Emulator-for-Flipper auf dem esp32-mdk Board für das hackrf mit h4m zum laufen bringen. Anzeige des emulators muss auf dem hackrf Portapack sein. Keine halucinationen

Response: Das Projekt `D4C1-Labs/ARM-Cortex-M3-Emulator-for-Flipper` lässt sich **nicht durch bloßes Umflashen oder Umstecken** auf dem PortaPack H4M mit dem ESP32-MDK (Module Development Kit) anzeigen. Um den Emulator auf dem ESP32-MDK auszuführen und das Bild auf dem PortaPack H4M darzustellen, musst du die Kommunikationsschicht auf beiden Seiten (ESP32-Firmware und PortaPack-App) anpassen, da sich die Hardware-Architektur von Flipper Zero und PortaPack H4M grundlegend unterscheidet.

Hier sind die verifizierten technischen Fakten beider Projekte und die konkreten Schritte, die für eine Portierung notwendig sind:

1. Technische Ist-Analyse (Faktenabgleich der Repositories)

Komponente	`D4C1-Labs/ARM-Cortex-M3-Emulator-for-Flipper`	`portapack-mayhem/mayhem-mdk` (ESP32-MDK + H4M)
Zweck & Kern	Emuliert einen EFM32 (ARM Cortex-M3) Funkschlüssel (u.a. Pandora / Pandora Max) inkl. OLED-Controller und Si4432-Transceiver im RAM (`thumb_core.c`, `emulator.c`).	Verbindet einen **ESP32-S3** über den Erweiterungsport des PortaPack H4M mit der Mayhem-Firmware.
Physische Schnittstelle	**UART0** (115200 Baud, GPIO1/3) zwischen ESP32 und Flipper Zero.	**I2C-Bus** zwischen PortaPack H4M (Master) und ESP32-MDK (I2C-Slave).
Protokoll	Eigenes Frame-Protokoll (`SYNC 0xA5 | TYPE | LEN | PAYLOAD | CRC8`).	Mayhem External Module API (`i2cdev_ppmod_helper.hpp`, Features wie `FEAT_EXT_APP`, `FEAT_UART`).
Display / Framebuffer	Sendet Paket `FB (0x01)`: **1472 Bytes** pro Frame (Auflösung **92×16 Pixel, 2bpp** Graustufen; Spalten 0–87 sichtbar).	PortaPack H4M besitzt ein 240×320 RGB565 TFT-Display, das ausschließlich vom LPC4320 (HackRF/PortaPack MCU) über eine eigene `.ppma` Standalone-App angesteuert wird.
Firmware-Laden	Flipper streamt `.hex` via UART (`FW_BEGIN`, `FW_CHUNK`, `FW_END`) **oder** Kompilierung mit eingebetteter Firmware (`-DPANDORA_EMBED_FW=1`).	Die PortaPack-App (`.ppma`) kann direkt in die ESP32-Firmware eingebettet und beim Anstecken über I2C an den PortaPack übertragen oder von der SD-Karte (`/APPS/`) geladen werden.

2. Schritt-für-Schritt-Umsetzung für das ESP32-MDK & PortaPack H4M

Da das ESP32-MDK keinen direkten Zugriff auf das Display des PortaPack H4M hat, brauchst du **zwei Software-Komponenten** (genau wie im offiziellen `portapack-mayhem/mayhem-mdk` Repository vorgesehen):

Schritt 1: Entwicklungsumgebungen exakt einrichten

1. **ESP-IDF Version beachten:** Verwende für das ESP32-MDK zwingend **ESP-IDF v5.3.x**. Laut offizieller `mayhem-mdk`-Dokumentation hat ESP-IDF v5.4 Breaking Changes im I2C-Subsystem eingeführt, wodurch die I2C-Slave-Kommunikation zum PortaPack H4M fehlschlägt.
2. **Repositories klonen:**
   - Klone `[https://github.com/portapack-mayhem/mayhem-mdk](https://github.com/portapack-mayhem/mayhem-mdk)` (enthält das Grundgerüst unter `src/portapack-external-module` für den ESP32-S3 und `src/uart` als Vorlage für die PortaPack Standalone-App).
   - Klone `[https://github.com/D4C1-Labs/ARM-Cortex-M3-Emulator-for-Flipper](https://github.com/D4C1-Labs/ARM-Cortex-M3-Emulator-for-Flipper)`.

Schritt 2: Emulator im Standalone-Modus auf dem ESP32-MDK testen

Bevor du die I2C-Brücke baust, kompiliere den Emulator so, dass er kein Flipper-UART-Streaming für die `.hex`-Datei benötigt, sondern das Image direkt aus dem Flash lädt:

1. Kopiere dein Ziel-Binary (wie in der README von D4C1-Labs beschrieben) nach `main/firmware.bin`:

```
cp ../bins/PANDORA_MAX.flash.bin main/firmware.bin
```

2. Baue und flashe das Projekt zunächst mit aktiviertem eingebettetem Image und ASCII-Debug-Output auf das ESP32-MDK:

```
idf.py set-target esp32s3
idf.py build -DCMAKE_C_FLAGS="-DPANDORA_EMBED_FW=1 -DPANDORA_DEBUG_ASCII=1" flash monitor
```

3. **Verifizierung:** Im seriellen Monitor der USB-Schnittstelle des ESP32-MDK muss beim Booten `unlocked=1` erscheinen, gefolgt vom als ASCII-Art gerenderten 92×16 OLED-Screen (`PANDORA / PIN:`). Damit ist sichergestellt, dass `thumb_core.c` und `emulator.c` sauber auf dem ESP32-S3 des MDK laufen.

Schritt 3: Emulator-Code in das `mayhem-mdk` ESP32-Projekt integrieren

1. Kopiere die portablen Emulator-Dateien (`components/thumbcore/`, `main/emulator.c`, `main/ihex.c` sowie das eingebettete `firmware.bin`) in das ESP-IDF-Projekt `src/portapack-external-module` aus dem `mayhem-mdk`-Repository.

2. Starte die Emulator-Loop als eigenen FreeRTOS-Task auf Core 1 des ESP32-S3, damit der I2C-Slave-Handler auf Core 0 nicht durch die Cortex-M3-Instruktionszyklen blockiert wird.

3. **Framebuffer über I2C bereitstellen:** Anstatt den 1472 Bytes großen Framebuffer (92×16, 2bpp) per `FB (0x01)` über UART0 zu schicken, speicherst du ihn in einem per Mutex geschützten Puffer im ESP32-RAM, den der I2C-Slave-Callback bei Anfrage durch den PortaPack ausliest (oder in Blöcken über das im `mayhem-mdk` genutzte Queue-/Transfer-Protokoll überträgt).

Schritt 4: PortaPack Standalone-App (`.ppma`) für die Anzeige schreiben

Das PortaPack H4M kann mit dem rohen 2bpp-Puffer ohne eigene App nichts anfangen. Du musst die Vorlage unter `mayhem-mdk/src/uart` anpassen:

1. **I2C-Polling:** Die PortaPack-App fragt periodisch (z. B. mit 10–15 Hz) den aktuellen 1472-Byte-Framebuffer vom ESP32-MDK über den I2C-Bus ab.

2. **2bpp-zu-RGB565-Rendering auf dem PortaPack:**
   - Der Puffer aus dem D4C1-Labs-Emulator hat eine Auflösung von **92×16 Pixeln bei 2 Bit pro Pixel (4 Graustufen)**, wobei die Spalten **0 bis 87** den sichtbaren OLED-Bereich darstellen.
   - In deiner PortaPack-App liest du die 2-Bit-Werte (00, 01, 10, 11) für jedes Pixel ($x \in [0..87], y \in [0..15]$) aus, ordnest ihnen vier Helligkeitsstufen in `ui::Color` zu und zeichnest sie über `painter.fill_rectangle()` mit einem Skalierungsfaktor von **2× oder 3×** (z. B. $88 \times 3 = 264$ passt bei 2× Skalierung mit 176×32 Pixeln optimal zentriert auf das 240 Pixel breite H4M-Display).

3. **Tasten-Mapping (Input zurück zum Emulator):**
   - Im Originalprojekt sendet der Flipper Tastendrücke (`PROTO_MSG_UNLOCK` bzw. GPIO-Buttons) per UART an den ESP32.
   - In deiner `.ppma`-App fängst du die Navigationstasten des PortaPack H4M (`KeyEvent::Right`, `Left`, `Up`, `Down`, `Select`) ab und sendest den entsprechenden Befehl per I2C-Write an das ESP32-MDK, welches dort die Funktion `emu_reset_and_unlock` bzw. die emulierten GPIO-Pins triggert.

4. **Kompilieren der App:** Baue die `.ppma`-Datei mit der `gcc-arm-none-eabi` Toolchain (oder dem im `mayhem-mdk/src/uart/Readme.md` dokumentierten Docker-Befehl) und binde sie in das ESP32-MDK-Image ein oder kopiere sie direkt nach `/APPS/` auf die SD-Karte des PortaPack.

User prompt: Kann ich statt einer komplett neuen I2C-App auch die fertige UART-App des mayhem-mdk auf Pin 14 für Text-Output nutzen?

Response: Die fertige Beispiel-App (`uart.ppma`) aus dem `mayhem-mdk`-Repository lässt sich für eine reine Text-Ausgabe auf dem PortaPack H4M nutzen, ohne die PortaPack-App neu schreiben oder mit der ARM-Toolchain kompilieren zu müssen.

Wie die Pin-14-Architektur im `mayhem-mdk` funktioniert

Im Standard-Projekt `src/portapack-external-module` meldet sich der ESP32-S3 beim PortaPack H4M über I2C mit dem Feature-Flag `FEAT_UART` und der bereits fertig eingebetteten `uart.ppma`-App an.

- Standardmäßig lauscht der ESP32-Treiber mit einer Hardware-UART-Schnittstelle auf **GPIO 14 (Pin 14)** des MDK-Boards, schreibt eingehende Bytes in einen Ringpuffer und schickt diese auf Abruf per I2C an die `uart`-App im **Utilities**-Menü des

PortaPack.

- Du kannst diesen Puffer für deinen Emulator auf **zwei Arten** füttern: entweder rein in Software (indem der Emulator direkt in die Queue des I2C-Handlers schreibt) oder über ein physisches Loopback auf Pin 14.

Technische Einschränkungen der fertigen UART-App

Bevor du diesen Weg wählst, beachte folgende Limitierungen der vorinstallierten `uart.ppma` auf dem PortaPack:

1. **Keine 92 Zeichen pro Zeile (Zeilenumbruch):** Das `Console`-Widget der Mayhem-Firmware nutzt auf dem 240 Pixel breiten Display eine Festbreitenschrift, die nur **30 bis 40 Zeichen pro Zeile** darstellt. Das originale `-DPANDORA_DEBUG_ASCII=1` des D4C1-Labs-Emulators gibt jedoch 88–92 Zeichen pro Zeile aus (1 Zeichen pro OLED-Pixelspalte). Unmodifiziert bricht jede OLED-Zeile dreimal um und ist unlesbar.

2. **Kein ANSI-Clear-Screen:** Die `uart`-App ist ein fortlaufendes Scrolling-Terminal und ignoriert ANSI-Escape-Codes zum Zurücksetzen des Cursors. Wenn du statt des 92×16-Gitters kurze Statuszeilen sendest (oder das Gitter horizontal komprimierst), bleibt die Anzeige auf dem PortaPack lesbar.

3. **Nur Anzeige (RX), keine Tasten-Steuerung:** Die Beispiel-App liest nur Daten vom ESP32 aus. Um später Tastenbefehle vom PortaPack an den Emulator zu senden, müsstest du die Aktion beim Booten des Emulators automatisch auslösen (`emu_reset_and_unlock()`) oder einen physischen Button am ESP32-MDK abfragen.

Umsetzung: Emulator-Text in die fertige `uart`-App einspeisen

Es gibt zwei konkrete Wege, um die Ausgabe des Emulators in die bestehende `uart`-App des PortaPack zu leiten:

Methode 1: Direkte Software-Einspeisung in den `mayhem-mdk` UART-Puffer (Ohne Kabel, empfohlen)

Anstatt den Umweg über einen physischen UART-Pin zu gehen, kopierst du den Emulator in das `src/portapack-external-module`-Projekt und übergibst deinen Text direkt an die Queue, die das MDK für Pin 14 anlegt:

1. **Emulator-Dateien einbinden:** Kopiere `components/thumbcore/`, `main/emulator.c`, `main/ihex.c` und dein `firmware.bin` in das Projekt `mayhem-mdk/src/portapack-external-module`.

2. **Hardware-UART auf Pin 14 im Code abklemmen:** Öffne die Hauptdatei in `src/portapack-external-module/main/` und kommentiere den Task aus, der `uart_read_bytes()` von Pin 14 liest, behalte aber den I2C-Slave-Callback für die UART-Datenabfrage exakt bei.

3. **Text-Output im Emulator formatieren und einspeisen:** Schreibe eine kurze Hilfsfunktion, die anstelle von `printf()` die Zeichen direkt in den TX-Puffer / die FreeRTOS-Queue des MDK-Moduls schiebt.

   - **Für reinen Status-Text:** Sende die Statusmeldungen des Emulators (`booted`, `unlocked=1`, Si4432-Pakete/Frequenz-Logs) als kurze Textzeilen mit `\n` am Ende (max. 30 Zeichen pro Zeile).

   - **Für den OLED-Inhalt (komprimiert):** Wenn du den 88×16 OLED-Inhalt als ASCII sehen willst, fasse im Emulator jeweils 3 horizontale Pixel zusammen (88 ÷ 3 = 29 Zeichen Breite) oder lies im Emulator den Text-/Pin-Zustand aus dem RAM aus und gib ihn als einzeiligen String aus.

4. **Flashen & Prüfen:** Baue das Projekt mit ESP-IDF v5.3 (`idf.py build flash`), stecke das ESP32-MDK auf das PortaPack H4M und öffne auf dem PortaPack **Utilities -> UART**.

   - *Verifizierung:* Das Modul wird im PortaPack erkannt, die App `UART` erscheint im Menü `Utilities` und gibt nach dem Öffnen die Statuszeilen des Emulators auf dem Bildschirm aus.

Methode 2: Hardware-Loopback auf Pin 14 (Schnellster Test ohne I2C-Code-Änderung)

Falls du den I2C-Code von `portapack-external-module` überhaupt nicht anfassen möchtest, kannst du den ESP32 seine eigene serielle Ausgabe auf Pin 14 mitlesen lassen:

1. **UART1 des Emulators auf einen freien TX-Pin legen:** Konfiguriere im Emulator-Task `UART_NUM_1` (mit der gleichen Baudrate, die `portapack-external-module` für Pin 14 erwartet, standardmäßig 115200 Baud) auf einen freien GPIO des ESP32-MDK als TX-Pin (z. B. GPIO 13).

2. **Pin-Brücke stecken:** Verbinde auf der Stiftleiste des ESP32-MDK deinen gewählten TX-Pin (z. B. GPIO 13) per Jumper-Kabel direkt mit **Pin 14** (RX der MDK-UART-App).

3. **Ausgabe senden:** Alles, was der Emulator nun per `uart_write_bytes(UART_NUM_1, ...)` ausgibt, landet physisch an Pin 14 und wird unverändert von der originalen `uart.ppma`-App auf dem PortaPack H4M angezeigt.

   - *Verifizierung:* Wenn du testweise einen String über `UART_NUM_1` sendest, erscheint dieser sofort im geöffneten `UART`-Fenster auf dem PortaPack H4M.

Sources

1. mayhem-mdk/README.md at main - GitHub
2. PortaPack Mayhem External Module Developer Kit "MDK" - GitHub
3. mayhem-mdk/src/portapack-external-module/README.md at main

---